
Flux Attention: Context-Aware Hybrid Attention for Efficient LLMs Inference

Quantong Qiu¹, Zhiyi Hong¹, Yi Yang¹, Haitian Wang¹, Kebin Liu², Qingqing Dang², Juntao Li^{1*}, Min Zhang¹

¹School of Computer Science and Technology, Soochow University

²Baidu Inc, China

Abstract

The quadratic computational complexity of standard attention mechanisms presents a severe scalability bottleneck for LLMs in long-context scenarios. While hybrid attention mechanisms combining Full Attention (FA) and Sparse Attention (SA) offer a potential solution, existing methods typically rely on static allocation ratios that fail to accommodate the variable retrieval demands of different tasks. Furthermore, head-level dynamic sparsity often introduces severe computational load imbalance and synchronization long-tails, which hinder hardware acceleration during autoregressive decoding. To bridge this gap, we introduce **Flux Attention**, a context-aware framework that dynamically optimizes attention computation at the layer level. By integrating a lightweight **Layer Router** into frozen pretrained LLMs, the proposed method adaptively routes each layer to FA or SA based on the input context. This layer-wise routing preserves high-fidelity information retrieval while ensuring contiguous memory access, translating theoretical computational reductions into practical wall-clock speedups. As a parameter-efficient approach, our framework requires only 12 hours of training on $8 \times A800$ GPUs. Extensive experiments across multiple long-context and mathematical reasoning benchmarks demonstrate that Flux Attention achieves a superior trade-off between performance and inference speed compared with baseline models, with speed improvements of up to $2.8\times$ and $2.0\times$ in the prefill and decode stages.

1 Introduction

Large Language Models (LLMs) have demonstrated strong capabilities in handling extended context windows for tasks such as document analysis, long-form reasoning, and question answering [26, 31]. However, the standard Full Attention (FA) mechanism [41] scales quadratically with sequence length, creating severe memory and computational bottlenecks during prefilling and autoregressive decoding. Sparse Attention (SA) mechanism addresses this by restricting computations to a subset of tokens to reduce the memory footprint [5, 51].

Modern architectures frequently employ hybrid attention mechanisms that integrate both FA and SA within a single network to balance inference efficiency and generation quality [52]. Conventional hybrid models typically rely on a static allocation of dense and sparse computation. However, downstream applications exhibit highly varied computational demands, as detailed in our preliminary study (Section 2.3). Retrieval-intensive tasks require dense token interactions to locate specific information, whereas context-holistic tasks focus on overarching semantics and remain stable under high sparsity [33]. Consequently, a static configuration risks performance degradation on retrieval tasks and wastes valuable computational resources on holistic tasks.

* Corresponding author: ljt@suda.edu.cn

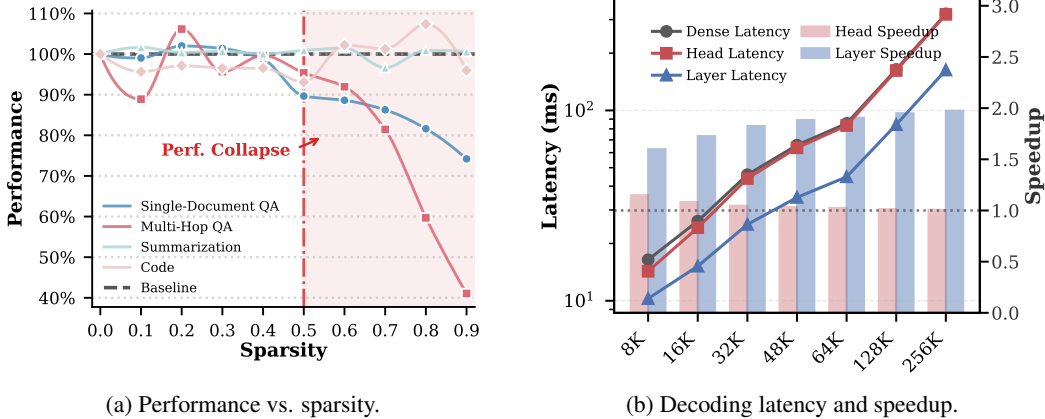


Figure 1: Impact of sparsity on performance and decoding efficiency. (a) Certain tasks suffer performance collapse beyond a specific threshold. (b) Layer-level sparsity achieves substantial decoding speedup, while head-level sparsity yields marginal speedup.

To achieve dynamic allocation, recent works [38] have explored fine-grained routing at the head level by assigning varying sparsity ratios to individual attention heads based on the input. While algorithmically flexible, this fine-grained routing introduces severe hardware inefficiencies during the memory-bandwidth-bound decode phase. Varying context lengths across heads lead to heterogeneous computational workloads within the same layer. This forces thread blocks executing sparse heads to idle while waiting for retrieval heads, creating a synchronization long-tail that prevents theoretical FLOP reductions from translating into actual wall-clock decoding speedups.

To overcome these challenges, we propose Flux Attention, a context-aware framework that dynamically optimizes attention computation at the layer level. Instead of managing individual heads, we introduce a lightweight Layer Router. By evaluating the semantic context of the input prompt, the router infers the underlying task demands and adaptively assigns each layer to FA or SA mode. This coarse granularity inherently preserves contiguous memory access, enabling the GPU to completely bypass the memory-intensive loading of historical KV tensors when SA is selected.

During training, we freeze all backbone LLM parameters and update only the lightweight Layer Router module. We employ a Gumbel-Softmax [17] relaxation for differentiable soft routing, allowing the model to smoothly learn the correlation between context complexity and computational budget. During inference, this soft formulation is discretized into deterministic hard routing, successfully translating theoretical computational savings into substantial wall-clock speedups.

Extensive evaluations on models such as Qwen-3 [49] and Llama-3.1 [12] demonstrate that Flux Attention successfully adapts sparsity levels across diverse tasks. Our parameter-efficient training converges in just 12 hours on an 8-GPU A800 node. Flux Attention achieves a superior performance-efficiency trade-off compared to existing baselines, delivering up to a 2.7x speedup during the prefill phase and a 2.0x acceleration during autoregressive decoding.

2 Preliminary

2.1 Functional Heterogeneity in Attention Mechanisms

During long-context inference, attention mechanisms in Large Language Models (LLMs) specialize functionally based on their sensitivity to historical context and computational demands. Specialized retrieval heads are essential for high-fidelity information recovery, as they precisely locate relevant tokens across extensive sequences [42]. UnComp [46] observe that heads with abnormally high entropy tend to aggregate at specific model depths to capture long-range dependencies. Layers dominated by these heads function as retrieval layers. To ensure precise retrieval, they require a Full Attention (FA) mode, where the Query (Q) interacts with all historical states Key (K) and Value (V):

$$\mathcal{O}_r = \text{Softmax}(QK^\top)V, \quad (1)$$

where the scaling factor is omitted for clarity. While FA preserves the complete context, its computational complexity is quadratic with sequence length N , posing challenges for efficient inference.

A substantial portion of heads instead focus on local semantic structures and are robust to context truncation. Layers predominantly composed of these sparse heads operate as sparse layers. Sparse layers employ a Sparse Attention (SA) mechanism to reduce computational overhead in long-sequence processing. SA optimizes efficiency by performing attention operations on a condensed subset of the most critical historical elements ($\tilde{K}$ and $\tilde{V}$):

$$\mathcal{O}_s = \text{Softmax} \left(Q\tilde{K}^\top \right) \tilde{V}. \quad (2)$$

2.2 Rethinking Hybrid Attention Mechanisms

To balance generation quality and inference efficiency, various hybrid attention mechanisms have been proposed. Existing methods, such as PruLong [4], DuoAttention [43], and LycheeDecode [25], adopt a static allocation strategy. They identify retrieval heads offline and permanently assign them full historical states, while uniformly sparsifying the context for the remaining heads across all tasks.

However, the demand for precise information retrieval varies depending on the specific task and input prompt. Elastic Attention [38] suggests dynamic, context-aware sparsity at the head level, which adjusts the retention of historical states dynamically. Although this fine-grained allocation optimizes the theoretical efficiency-performance trade-off, it yields limited actual decoding acceleration. The dynamic adjustment at the head level introduces significant system-level overhead and irregular memory access patterns during deployment, limiting the achievable speedup during the decode phase.

2.3 Motivational Observations

To investigate the limitations of existing sparsity mechanisms, we formalize the quantification of model-level sparsity. The Model Sparsity Ratio (Ω_{MSR}) quantifies the overall proportion of sparse attention mechanisms applied across the model:

$$\Omega_{\text{MSR}} = \frac{1}{H \times L} \sum_{\ell=1}^L \sum_{h=1}^H \mathbb{I}[\pi^{(\ell,h)} = \text{SA}], \quad (3)$$

where $\pi^{(\ell,h)}$ denotes the assigned attention mode (FA or SA) for head h in layer ℓ , and $\mathbb{I}[\cdot]$ is the indicator function.

Settings To investigate the impact of varying sparsity ratios (Ω_{MSR}) on long-context LLMs, we profile task accuracy and decode latency. For the accuracy evaluation in Figure 1(a), we use a matrix entropy metric based on UnComp [46] to quantify the information density of individual layers. We rank the layers using these calculated entropy scores and progressively replace the lowest-scoring ones with SA. Model performance is then evaluated across real-world tasks from LongBench [1]. For hardware efficiency (Figure 1(b)), we compare the decode latency and achievable speedup of our layer-level sparsity against a static head-level sparsity baseline. Appendix C provides details on the entropy scoring formulation and latency measurement implementation.

Results Our analysis reveals two bottlenecks in current hybrid attention mechanisms. First, as shown in Figure 1(a), model performance does not degrade linearly with increasing Ω_{MSR} . Instead, accuracy drops sharply for retrieval-intensive tasks once a specific sparsity threshold is exceeded. This indicates that static sparsity assignments do not adapt to varying contextual demands, necessitating a context-aware dynamic retention strategy for historical states. Second, Figure 1(b) demonstrates a distinct discrepancy in hardware efficiency. While head-level sparsity provides algorithmic flexibility, it introduces severe hardware bottlenecks during the memory-bandwidth-bound decode phase. It creates a severe synchronization long-tail effect. Thread blocks executing sparse heads finish quickly but must idle while waiting for memory-intensive retrieval heads within the same layer. This intra-layer load imbalance yields only marginal wall-clock speedups. In contrast, layer-level sparsity ensures uniform computational workloads across all thread blocks. By completely bypassing historical KV loading for designated layers, it eliminates synchronization stalls, effectively translating theoretical FLOP reductions into substantial decode acceleration.

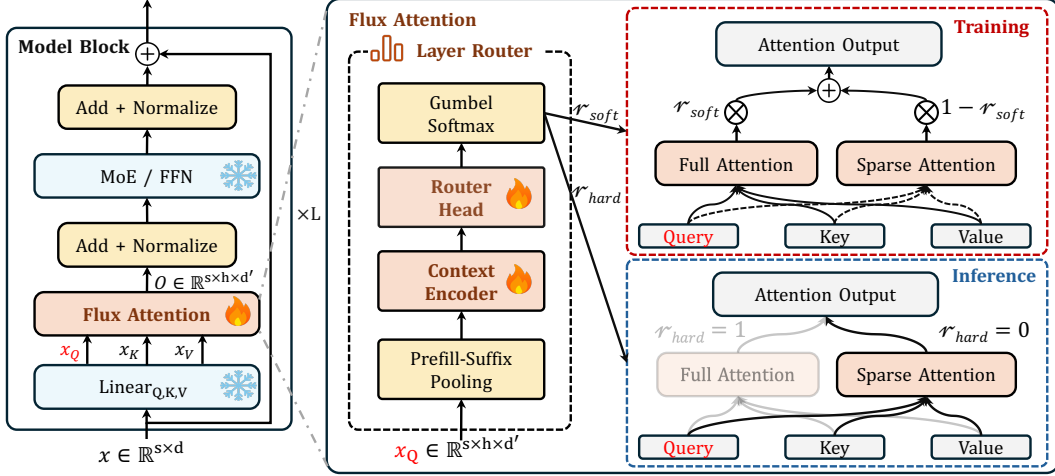


Figure 2: Overview of our dynamic layer-level routing architecture. The model incorporates a Layer Router that assigns each layer to either FA or SA based on the input query x_Q .

These observations present a fundamental dilemma. *Fine-grained head-level sparsity is hardware-unfriendly during decode, whereas static sparsity risks performance collapse.* To address this, we propose a dynamic, context-aware hybrid attention mechanism operating at the layer level to balance model performance with inference efficiency.

3 Methodology

We introduce a Flux Attention mechanism to address the hardware inefficiencies of fine-grained sparsity and the rigidity of static allocations. As illustrated in Figure 2, our architecture relies on a dynamic Layer Router that adaptively assigns each layer to either FA or SA based on the input query. This approach is parameter-efficient: the original LLM backbone parameters remain strictly frozen during training. Optimization only updates the lightweight components of the Layer Router, ensuring rapid convergence while preserving pre-trained weights.

3.1 Context-Aware Layer Router Design

Within the Flux Attention module, a lightweight Layer Router determines the optimal attention mechanism for a given context.

Architecture and Feature Extraction As shown in Figure 2, the router receives the incoming query tensor $x_Q \in \mathbb{R}^{s \times h \times d'}$ as input, where s represents the sequence length, h denotes the number of heads, and d' indicates the head dimension. To efficiently extract semantic context, we apply a Prefill-Suffix Pooling operation to x_Q to extract representations of the initial and final prompt tokens. This operation efficiently aggregates the token-level features into a single sequence-level descriptor. Subsequently, a Context Encoder (MLP) processes this pooled representation to capture contextual dependencies, after which a Router Head (MLP) projects these features into unnormalized routing logits, denoted as π_{FA} and π_{SA} .

Differentiable Training via Soft Routing Optimizing the Layer Router is challenging because the binary routing decisions are discrete and non-differentiable. To address this, we apply the Gumbel-Softmax relaxation [17] to enable end-to-end backpropagation. During training, we sample continuous routing weights $r_{\text{soft}} \in (0, 1)$, which represent the probability of selecting the FA mechanism. This computation is defined as follows:

$$r_{\text{soft}} = \frac{\exp((\pi_{FA} + g_{FA})/\tau)}{\exp((\pi_{FA} + g_{FA})/\tau) + \exp((\pi_{SA} + g_{SA})/\tau)} \quad (4)$$

where $g_{\text{FA}}, g_{\text{SA}} \sim \text{Gumbel}(0, 1)$ are independent and identically distributed samples drawn from the Gumbel distribution, and $\tau > 0$ denotes the temperature parameter. The output of the Flux Attention layer is then computed as a convex combination:

$$\mathcal{O}_{\text{train}} = r_{\text{soft}} \cdot \text{FA}(Q, K, V) + (1 - r_{\text{soft}}) \cdot \text{SA}(Q, \tilde{K}, \tilde{V}). \quad (5)$$

The temperature τ controls the smoothness of the routing distribution. We employ a temperature annealing schedule to minimize the train-test discrepancy. Initially, τ is set to a high value to encourage exploration and ensure smooth gradient flow. As training progresses, τ linearly decays towards a small value.

Deterministic Inference via Hard Routing During the inference phase, the router outputs a binary decision $r_{\text{hard}} \in \{0, 1\}$ using an $\arg \max$ operation over the generated logits. When $r_{\text{hard}} = 0$, the layer executes the SA mechanism.

3.2 Training Objective and Sparsity Constraint

We formulate the training objective as a constrained optimization problem to balance generation quality and computational efficiency. Without intervention, the router tends to degenerate by sending all queries to the FA mode, which trivially minimizes the language modeling loss.

A dynamic penalty mechanism controls the inference budget. Let $t \in (0, 1)$ denote the target computational budget for sparse computation (i.e., the permissible fraction of SA layers, corresponding to $1 - \Omega_{\text{MSR}}$). Notably, instead of enforcing a rigidly fixed t for each task, we impose *task-dependent non-tight constraints* with predefined lower and upper bounds, since the optimal sparsity for a given task is inherently unknown. We therefore define the sparsity deviation as $L_{\text{diff}}(\mathcal{X}) = \mathbb{E}_{\mathcal{X}}[1 - r_{\text{soft}}] - t$, which represents the gap between the expected sparse routing probability across all layers and the allocated budget. We solve the overall optimization objective via Lagrangian relaxation:

$$\max_{\lambda_1, \lambda_2 \geq 0} \min_{\theta} \underbrace{L_{\text{language}}(\mathcal{X})}_{\text{language modeling}} + \underbrace{\lambda_1 L_{\text{diff}}(\mathcal{X}) + \lambda_2 L_{\text{diff}}^2(\mathcal{X})}_{\text{sparsity regularization}}, \quad (6)$$

where θ represents the trainable parameters of the Layer Router, and $L_{\text{language}}(\mathcal{X})$ is the standard cross-entropy loss. The Lagrangian multipliers λ_1 and λ_2 are task-specific trainable Lagrange multipliers optimized via gradient ascent [4], which decouple the sparsity–performance trade-offs across tasks and mitigate optimization conflicts.

3.3 Efficient Deployment

To translate theoretical sparsity gains into real-world inference acceleration and memory savings, Flux Attention decouples routing computation between the prefill and decode phases, with a sparse-decode implementation aligned with our experimental settings.

The Layer Router infers only once during the prefill phase, generating a deterministic hard routing decision ($r_{\text{hard}} \in \{0, 1\}$) per layer based on the input context. This decision is cached and reused across all decoding steps, eliminating per-token routing overhead. Our sparse-decode configuration further optimizes efficiency: for sparse layers, we only maintain the minimal KV cache required by the sparse kernel, fully bypassing full historical KV access and storage; for retrieval layers, complete KV cache is retained to preserve retrieval performance. This design delivers significant decoding speedups and KV cache reduction in long-context scenarios.

4 Experiments

4.1 Settings

Training and Data We select Qwen3 (4B and 8B) [49] and Llama-3.1-8B-Instruct [12] as the backbone LLMs. We construct the training dataset by combining five sources: ChatQA2-Long-SFT-data [47], MuSiQue [40], CoLT-132K [22], GovReport [16], and XSum [32]. This dataset covers both retrieval-intensive tasks (Single-Doc QA and Multihop QA) and context-holistic tasks (code completion, summarization, and in-context learning). The resulting dataset spans sequence

Table 1: Performance on LongBench-E [1]. We report average performance (Perf.) and Ω_{MSR} per task category. The 1st and the 2nd performance in each comparison group are highlighted with **bold font** and underlined, respectively. Gray-shaded rows denote the sparse-decode configuration.

Method	S-Doc QA		M-Doc QA		Summ		In-Context			Synthetic		Code		Avg.	
	Qasper	MF-en	HotQA	2Wiki	Gov.	M.News	TREC	TQA	SAMS	PCount	Pre	RB-P	Lcc	Perf.	Ω_{MSR}
Qwen3-4B backbone model															
Qwen3-4B	35.21	52.16	44.81	32.15	33.47	23.45	70.67	88.22	39.74	2.33	96.84	50.84	57.93	48.45	-
+ DuoAttention	35.83	49.84	47.09	32.24	<u>33.32</u>	23.70	69.33	85.87	39.75	4.50	<u>94.57</u>	50.56	57.43	48.22	0.50
+ PruLong	34.15	50.78	44.48	<u>32.89</u>	32.96	23.53	67.67	88.69	39.55	3.17	90.17	49.00	54.07	47.16	0.50
+ TriangleMix	35.55	52.02	45.37	31.76	<u>33.32</u>	23.70	69.00	88.20	39.74	3.83	91.51	48.58	56.38	47.72	0.50
+ FluxAttn (FA-SSA)	35.02	49.44	<u>49.64</u>	32.27	33.26	23.48	69.33	<u>88.29</u>	39.78	1.50	94.56	53.44	<u>59.69</u>	<u>48.72</u>	0.44
+ FluxAttn (FA-XA)	<u>35.74</u>	<u>51.70</u>	45.83	32.34	33.57	<u>23.66</u>	69.00	87.23	39.81	3.50	93.74	50.81	59.28	48.32	0.53
+ FluxAttn (FA-TA)	35.02	50.89	45.17	34.24	33.02	23.53	<u>69.00</u>	88.08	40.38	<u>3.94</u>	96.06	<u>51.68</u>	60.00	48.76	0.47
+ FluxAttn (FA-SSA)	35.10	51.68	49.65	32.86	33.04	23.42	69.33	88.00	<u>40.00</u>	1.67	94.47	51.40	58.68	48.59	0.44
Qwen3-8B backbone model															
Qwen3-8B	41.22	49.92	58.98	44.21	33.27	23.42	71.33	86.77	41.83	2.00	98.33	56.08	66.31	52.16	-
+ DuoAttention	41.78	51.55	55.96	41.70	33.24	23.34	69.33	89.35	41.62	0.50	98.93	57.54	<u>69.39</u>	52.13	0.50
+ PruLong	37.95	51.20	51.94	36.48	33.11	23.36	69.00	87.90	<u>42.11</u>	1.00	98.00	57.05	67.66	50.80	0.50
+ TriangleMix	40.82	<u>51.31</u>	57.57	44.51	33.32	23.35	71.33	86.73	41.79	2.00	94.33	55.04	65.89	51.65	0.50
+ FluxAttn (FA-SSA)	40.30	50.49	56.02	40.90	33.01	23.55	<u>71.67</u>	88.31	41.61	0.33	100.00	59.46	68.27	<u>52.18</u>	0.46
+ FluxAttn (FA-XA)	40.41	50.26	<u>57.78</u>	40.57	<u>33.27</u>	<u>23.51</u>	69.67	87.19	42.12	<u>1.33</u>	99.33	55.41	65.51	51.57	0.51
+ FluxAttn (FA-TA)	<u>41.00</u>	49.76	58.19	<u>44.36</u>	33.32	23.35	70.00	<u>88.77</u>	41.70	<u>1.33</u>	<u>99.67</u>	55.60	67.22	52.22	0.47
+ FluxAttn (FA-SSA)	39.92	50.04	55.72	40.81	33.03	23.50	72.00	88.48	40.96	0.33	99.22	<u>58.57</u>	69.46	52.05	0.46
Llama-3.1-8B-Instruct backbone model															
Llama-3.1-8B-Instruct	44.06	53.44	59.62	44.08	34.50	26.02	71.00	90.54	42.94	12.67	99.33	47.78	63.85	53.28	-
+ DuoAttention	34.63	50.74	49.70	36.41	34.25	25.78	70.00	91.45	42.13	9.80	97.33	53.59	68.55	52.11	0.50
+ PruLong	41.51	52.36	50.46	37.57	34.25	25.86	66.33	89.93	41.72	9.07	97.00	56.84	<u>66.23</u>	51.68	0.50
+ TriangleMix	<u>45.10</u>	54.60	56.67	41.88	34.09	25.51	<u>71.33</u>	90.93	<u>42.63</u>	<u>10.62</u>	94.67	43.64	59.35	51.67	0.50
+ FluxAttn (FA-SSA)	45.25	<u>54.42</u>	54.54	41.34	<u>34.54</u>	26.16	68.33	91.91	42.17	9.00	97.67	47.74	65.35	52.28	0.51
+ FluxAttn (FA-XA)	42.14	53.13	58.53	43.50	34.66	26.06	70.67	91.46	43.13	8.00	99.67	50.91	64.78	53.07	0.72
+ FluxAttn (FA-TA)	44.77	54.12	57.35	<u>43.43</u>	34.31	25.80	72.33	91.32	42.62	9.33	98.33	45.48	60.70	<u>52.42</u>	0.62
+ FluxAttn (FA-SSA)	43.76	53.41	<u>57.36</u>	39.43	32.96	25.63	70.33	91.27	42.20	11.00	<u>98.67</u>	45.60	66.17	52.30	0.51

lengths ranging from 1K to 64K tokens, and contains approximately 0.74B tokens in total. For the context-holistic and retrieval-intensive task categories, we empirically set $t = 1.0$ and $t = 0.45$, respectively, as motivated by Section 2.3. We conduct the training process using eight A800 GPUs, and each run completes within 12 hours. We provide additional training details in the Appendix D and list the hyperparameters in the table 3.

Evaluation We compare our method with representative sparsity approaches: DuoAttention [44], PruLong [4], and TriangleMix [14]. The computation modes for sparse layer attention include Streaming Sparse Attention (SSA) [45], XAttention (XA) [48], and Triangle Attention (TA). The configurations for layer computation follow the format of “{Retrieval Layer mode}-{Sparse Layer mode}” (e.g., FA-SSA denotes the use of FA for retrieval layers and SSA for sparse layers). All evaluations are conducted using the L00M-Eval framework [39].

4.2 Evaluation Results

Real-world Long-context Tasks Table 1 presents the evaluation results on LongBench-E [1], a real-world long-context benchmark that comprises 14 tasks across 6 categories with varying context lengths. FluxAttn maintains the performance of the model on long-context tasks while achieving substantial context compression. Across the Qwen3 series, variants of FluxAttn frequently match or slightly exceed the average performance of the full attention baselines. We further evaluate the effect of applying sparse attention during the decode phase, as indicated in the shaded rows. The method remains competitive under sparse decode. On Qwen3-4B, the sparse-decode configuration achieves an average score of 48.59, which remains above the full attention baseline. For Qwen3-8B and Llama-3.1-8B-Instruct, the average scores (52.05 and 52.30, respectively) demonstrate only a slight degradation compared to the standard dense decoding approach.

Length Extrapolation Capability Testing To further assess the ability of the models to handle extreme context lengths, we evaluated our method on the RULER benchmark [15], which tests length extrapolation capabilities from 8K to 256K tokens. The results are summarized in Table 2. Overall, FluxAttn demonstrates robust length extrapolation, maintaining information retrieval and reasoning capabilities even at the 256K context boundary, where many existing sparse attention baselines experience severe performance degradation. Consistent with our findings in real-world tasks, we also observe that extending sparsity to the decode phase (shaded rows) preserves the extrapolation

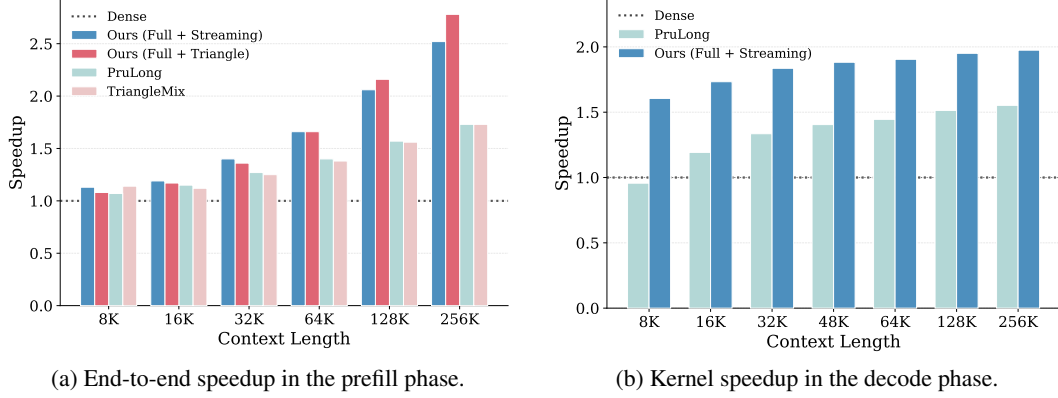


Figure 3: Speedup comparison across different context lengths. The dotted line represents the dense baseline performance (1.0x).

capabilities. The sparse-decode configuration of FluxAttn on Qwen3-4B achieves an average score of 67.19 (the highest among all methods in the comparison group) and a score of 56.00 at 256K. This result further validates that our method can achieve comprehensive efficiency gains without compromising ultra-long context understanding.

Long-form Reasoning and Math Tasks We further evaluate our models on the long-context reasoning benchmark LongBench-V2 [2], as well as the mathematical reasoning tasks GSM8K [6] and AIME24 [30]. Table 2 demonstrates that FluxAttn exhibits strong performance across both domains. On LongBench-V2, the proposed method attains the highest scores on both the easy and hard subsets among all baselines. Furthermore, our approach improves the performance on the mathematical benchmarks, yielding the best results on GSM8K and AIME24. This proves that FluxAttn robustly preserves complex logical reasoning capabilities.

4.3 Overall Inference Efficiency

To evaluate the hardware acceleration of our method, we benchmark the inference speedup of FluxAttn against the standard dense baseline and existing sparse methods across varying context lengths. Figure 3 presents the speedup metrics for both the prefill and decode phases.

End-to-End Prefill Acceleration Figure 3(a) shows the end-to-end latency reduction during the compute-bound prefill phase. As the context window expands, the quadratic complexity of standard attention becomes a bottleneck, allowing our dynamic routing mechanism to demonstrate substantial gains. At a 256K context length, our method (configured with Full + Triangle) achieves up to a $2.8\times$ end-to-end speedup, outperforming static baselines such as PruLong and TriangleMix.

Kernel-Level Decode Acceleration The advantage of layer-level routing is evident during the memory-bandwidth-bound decode phase, as shown in Figure 3(b). While prior approaches like PruLong struggle to translate theoretical sparsity into proportional wall-clock speedups due to fragmented memory access, FluxAttn solves this bottleneck by operating at the layer level. Our method achieves a scalable kernel speedup, approaching $2.0\times$ at a 256K context length. This result empirically shows that our context-aware, layer-wise routing aligns with modern GPU execution patterns to deliver improved inference efficiency.

Router Overhead Analysis A critical requirement for dynamic routing is minimizing its own computational cost. As illustrated in Figure 9, our router incurs a negligible overhead, averaging only 0.20 ms per layer. Notably, the design exhibits length-invariant stability, maintaining a constant execution speed across sequence lengths ranging from 512 to 1M tokens. This ensures that the routing mechanism itself does not become a bottleneck at extreme context lengths, thereby preserving the substantial speedups achieved in the prefill phase.

Table 2: Model performance on RULER [15], LongBench-v2 [2] and some Math tasks [6, 30]

Models	RULER							LongBench-v2			Math		
	8K	16K	32K	64K	128K	256K	Perf.	Easy	Hard	Perf.	GSM8K	AIME24	Perf.
Qwen3-4B backbone model													
Qwen3-4B	87.49	86.82	60.05	70.98	53.19	43.27	66.00	32.67	22.18	25.96	39.70	30.35	35.03
+ DuoAttention	79.38	76.08	52.91	69.02	43.28	44.96	60.67	31.33	24.06	26.68	39.70	37.05	38.38
+ PruLong	74.21	75.72	47.88	59.27	47.10	45.69	60.25	28.00	25.56	26.44	39.70	30.35	35.03
+ TriangleMix	87.42	85.10	58.73	67.94	50.97	44.47	63.74	31.33	22.18	25.48	40.30	37.25	38.78
+ FluxAttn (FA-SSA)	81.58	82.11	58.73	72.89	52.81	56.91	66.95	29.33	28.57	28.85	40.30	37.05	38.68
+ FluxAttn (FA-XA)	86.79	84.94	59.52	68.82	51.77	43.43	63.67	30.00	24.06	26.20	42.20	40.35	41.28
+ FluxAttn (FA-TA)	84.28	84.53	60.58	68.60	51.91	51.64	65.55	31.33	26.32	28.12	45.00	40.35	42.68
+ FluxAttn (FA-SSA)	80.36	80.75	56.08	71.49	59.17	56.00	67.19	28.00	28.20	28.12	39.90	37.25	38.58
Qwen3-8B backbone model													
Qwen3-8B	89.69	85.62	63.23	82.39	65.84	66.71	75.74	39.33	27.82	31.97	40.60	32.35	36.48
+ DuoAttention	86.68	86.01	63.23	77.52	61.50	61.95	72.41	40.67	25.56	31.01	41.20	35.65	38.43
+ PruLong	83.85	80.86	60.05	77.25	62.54	61.49	70.97	36.00	28.20	31.01	40.40	32.35	36.38
+ TriangleMix	81.01	75.67	63.49	73.76	61.54	66.84	70.47	36.00	27.44	30.53	41.20	44.15	42.68
+ FluxAttn (FA-SSA)	84.09	81.90	60.58	79.30	64.74	65.27	73.03	36.67	29.32	31.97	46.90	42.35	44.63
+ FluxAttn (FA-XA)	85.88	85.54	65.08	81.95	65.09	65.38	74.65	32.67	32.71	32.69	43.20	35.65	39.43
+ FluxAttn (FA-TA)	87.49	86.17	60.85	78.72	60.75	63.03	73.51	37.33	27.44	31.01	43.00	39.05	41.03
+ FluxAttn (FA-SSA)	83.54	81.00	59.79	77.93	64.83	65.12	72.51	39.33	28.20	32.21	45.30	43.20	44.25
Llama-3.1-8B-Instruct backbone model													
Llama-3.1-8B-Instruct	92.88	92.83	89.46	70.79	80.12	72.34	83.47	32.00	33.08	32.69	42.30	30.35	36.33
+ DuoAttention	91.71	86.35	85.65	62.65	62.30	38.69	70.33	26.67	28.57	27.88	44.40	33.65	39.03
+ PruLong	86.96	76.55	70.65	54.52	48.18	30.00	59.87	30.00	24.44	26.44	41.30	29.85	35.58
+ TriangleMix	92.44	90.76	86.75	68.00	78.25	64.39	80.46	29.33	25.56	26.92	46.30	37.05	41.68
+ FluxAttn (FA-SSA)	82.88	78.09	70.39	52.29	62.20	50.73	76.75	34.00	28.95	30.77	45.30	37.05	41.18
+ FluxAttn (FA-XA)	92.43	90.85	88.23	68.56	75.86	60.80	79.51	36.00	31.95	33.41	44.40	33.65	39.03
+ FluxAttn (FA-TA)	92.72	90.53	86.45	67.78	80.63	67.09	81.50	34.67	28.95	31.01	46.90	38.30	42.60
+ FluxAttn (FA-SSA)	90.11	79.39	79.22	56.08	62.94	59.39	73.67	34.67	30.08	31.73	45.90	37.35	41.63

5 Analysis

5.1 Dynamic Allocation Strategy of the Layer Router

Task-Level Dynamic Sparsity Different downstream tasks impose inherently distinct requirements on attention sparsity. As shown in the upper region of Figure 4, retrieval-intensive tasks frequently activate FA (dark blue) to support the dense token interactions required for fact-finding. Conversely, context-holistic tasks predominantly route the mid-to-high layers to SA, which validates that high-level holistic semantic understanding is highly robust to attention sparsification. This demonstrates that Flux Attention replaces static allocations with task-aware dynamic sparsity.

Context-Aware Intra-Task Sparsity Beyond cross-task adaptation, the router further captures the intrinsic sparsity requirements of individual input contexts, rather than merely memorizing coarse-grained task-level patterns. This instance-level variance is evident where intermediate activation frequencies ($\sim 0.4 - 0.6$, light blue) within a single task show the router adjusting to the complexity of different inputs. We also find that specific layers (e.g., layers 0, 1, 5, 13, and 15–17) are consistently routed to FA across all tasks. This indicates the router preserves the universal architectural properties of the backbone while allocating the remaining computational budget based on specific task and context demands.

Notably, the emergence of this fine-grained, task-aware routing relies on a well-balanced training curriculum. An unbalanced data distribution can cause the router to collapse into a homogenized routing strategy, as extensively analyzed in Appendix E.1. Furthermore, we find that a prefill-suffix pooling operation on the boundary 100 tokens is highly effective in driving this context-aware routing, as it isolates essential instruction signals from sequence noise (detailed in Appendix E.2).

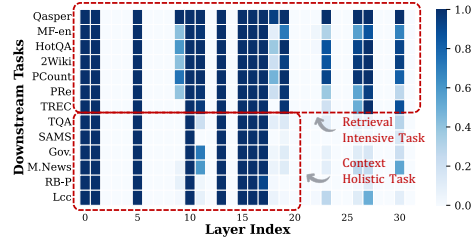


Figure 4: Overview of the layer-wise routing activation frequencies in Llama-3.1-8B-Instruct. Dark blue indicates layers consistently routed to FA across all six tasks in LongBench-E, whereas light blue denotes layers consistently routed to SA.

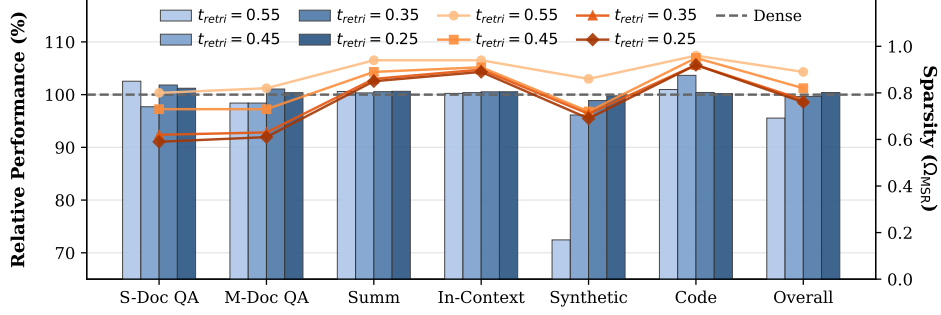


Figure 5: Comparison of performance and test-time Ω_{MSR} among different training sparsity target t settings. The bar chart denotes the performance and the line chart denotes Ω_{MSR} in each task.

5.2 Impact of Target Sparsity Allocation

We study the impact of target sparsity t on model performance. Specifically, we fix the target sparsity of context-holistic tasks to 1, while progressively decreasing target sparsity for retrieval-intensive tasks (t_{retri}) from 0.55 to 0.25. As shown in Figure 5, decreasing t_{retri} causes the resulting (Ω_{MSR}) allocated by the model exhibits slightly greater task-level differentiation across different tasks. However, Ω_{MSR} does not strictly match the target t . This discrepancy arises because we use *task-dependent and non-tight constraints*, which do not force the model to exactly satisfy the prescribed sparsity. We provide full training curves and further explanations in Appendix E.3. Additionally, when t_{retri} is set too low (e.g., 0.25) to allocate a higher proportion of FA computation, the overall performance can even surpass that of the backbone model. Conversely, setting t_{retri} too high causes the performance on retrieval-intensive tasks to drop sharply, consistent with the observations in Section 2.3. To optimize inference efficiency, we adopt $t_{\text{retri}} = 0.45$ in our main experiments, which achieves a favorable balance between strong overall performance and computational cost.

5.3 Scalability via Backbone Adaptation

To evaluate the flexibility of Flux Attention, we investigate how well the method supports continued training. A critical question for dynamic sparsity methods is whether the routing mechanism can be decoupled from the backbone for subsequent model adaptation. To test this, we freeze the weights of the trained Layer Router, which fixes its learned dynamic allocation strategy, and continue training the model backbone using the data mixture from Section 4.1.

As Figure 6 illustrates, continued training yields steady performance improvements across different models. Notably, both Qwen3-8B and Qwen3-4B rapidly surpass their original backbone performance (dashed lines) within just 50 steps and maintain a significant gain. While Llama3.1-8B-Instruct initially falls below its baseline, it demonstrates strong and continuous recovery throughout the training process, steadily closing the performance gap. We attribute this delayed convergence to the heightened sensitivity of instruction-tuned models, which require additional steps to realign their complex representations under forced sparsity constraints. These trends indicate that the backbone can effectively adapt its representations to the prescribed sparse pathways. Flux Attention thus offers practical post-training flexibility, allowing users to lock in an efficiency budget and fine-tune for downstream applications without disrupting the routing dynamics.

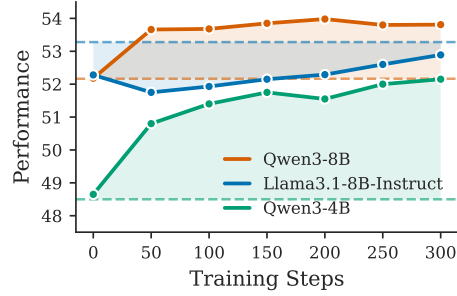


Figure 6: Performance trajectories during continued training with a frozen Layer Router. The backbone effectively adapts its representations to the established sparse pathways, demonstrating steady improvement over time.

6 Conclusion

We introduce *Flux Attention*, a context-aware dynamic routing framework mitigating the quadratic computational bottleneck of Large Language Models in long-context scenarios. Unlike existing hybrid attention mechanisms relying on rigid static allocations or hardware-inefficient head-level routing, our approach employs a lightweight Layer Router adaptively assigning each transformer layer to full or sparse Attention based on task and input demands. Extensive evaluations demonstrate our parameter-efficient method, requiring only 12 hours of training, achieves speedups up to $2.8\times$ during prefilling and $2.0\times$ during autoregressive decoding. Crucially, it preserves high-fidelity information recovery across diverse long-context benchmarks, establishing a superior and scalable trade-off between generation quality and inference efficiency for modern LLMs.

References

- [1] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3119–3137, Bangkok, Thailand, August 2024. Association for Computational Linguistics.
- [2] Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, Shulin Cao, Jiazheng Xu, Lei Hou, Yuxiao Dong, et al. Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3639–3664, 2025.
- [3] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv:2004.05150*, 2020.
- [4] Adithya Bhaskar, Alexander Wettig, Tianyu Gao, Yihe Dong, and Danqi Chen. Cache me if you can: How many kvs do you need for effective long-context lms? *arXiv preprint arXiv:2506.17121*, 2025.
- [5] Rewon Child. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.
- [6] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021.
- [7] Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality, 2024.
- [8] DeepSeek-AI. Deepseek-v3.2-exp: Boosting long-context efficiency with deepseek sparse attention, 2025.
- [9] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity, 2022.
- [10] Yizhao Gao, Zhichen Zeng, Dayou Du, Shijie Cao, Hayden Kwok-Hay So, Ting Cao, Fan Yang, and Mao Yang. Seerattention: Learning intrinsic sparse attention in your llms. *arXiv preprint arXiv:2410.13276*, 2024.
- [11] Paolo Glorioso, Quentin Anthony, Yury Tokpanov, James Whittington, Jonathan Pilault, Adam Ibrahim, and Beren Millidge. Zamba: A compact 7b ssm hybrid model, 2024.
- [12] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [13] Junxian Guo, Haotian Tang, Shang Yang, Zhekai Zhang, Zhijian Liu, and Song Han. Block Sparse Attention. <https://github.com/mit-han-lab/Block-Sparse-Attention>, 2024.
- [14] Zhiyuan He, Yike Zhang, Chengruidong Zhang, Huiqiang Jiang, Yuqing Yang, and Lili Qiu. Trianglemix: Accelerating prefilling via decoding-time contribution sparsity, 2025.
- [15] Cheng-Ping Hsieh, Simeng Sun, Samuel Krman, Shantanu Acharya, Dima Rekish, Fei Jia, Yang Zhang, and Boris Ginsburg. Ruler: What’s the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024.

- [16] Luyang Huang, Shuyang Cao, Nikolaus Parulian, Heng Ji, and Lu Wang. Efficient attentions for long document summarization. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1419–1436, Online, June 2021. Association for Computational Linguistics.
- [17] Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with gumbel-softmax. *arXiv preprint arXiv:1611.01144*, 2016.
- [18] Xiaodong Ji, Hailin Zhang, Fangcheng Fu, and Bin Cui. Sale : Low-bit estimation for efficient sparse attention in long-context llm prefilling, 2025.
- [19] Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H Abdi, Dongsheng Li, Chin-Yew Lin, et al. Minference 1.0: Accelerating pre-filling for long-context llms via dynamic sparse attention. *Advances in Neural Information Processing Systems*, 37:52481–52515, 2024.
- [20] Jerome Ku, Eric Nguyen, David W. Romero, Garyk Brix, Brandon Yang, Anton Vorontsov, Ali Taghibakhshi, Amy X. Lu, Dave P. Burke, Greg Brockman, Stefano Massaroli, Christopher Ré, Patrick D. Hsu, Brian L. Hie, Stefano Ermon, and Michael Poli. Systems and algorithms for convolutional multi-hybrid language models at scale, 2025.
- [21] Xunhao Lai, Jianqiao Lu, Yao Luo, Yiyuan Ma, and Xun Zhou. Flexprefill: A context-aware sparse attention mechanism for efficient long-sequence inference. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [22] Jia Li, Hao Zhu, Huanyu Liu, Xianjie Shi, He Zong, Yihong Dong, Kechi Zhang, Siyuan Jiang, Zhi Jin, and Ge Li. aixcoder-7b-v2: Training llms to fully utilize the long context in repository-level code completion. *arXiv preprint arXiv:2503.15301*, 2025.
- [23] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. Snapkv: Llm knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024.
- [24] Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meirom, Yonatan Belinkov, Shai Shalev-Shwartz, Omri Abend, Raz Alon, Tomer Asida, Amir Bergman, Roman Glozman, Michael Gokhman, Avashalom Manevich, Nir Ratner, Noam Rozen, Erez Shwartz, Mor Zusman, and Yoav Shoham. Jamba: A hybrid transformer-mamba language model, 2024.
- [25] Gang Lin, Dongfang Li, Zhuoen Chen, Yukun Shi, Xuhui Chen, Baotian Hu, and Min Zhang. Lycheedecode: Accelerating long-context LLM inference via hybrid-head sparse decoding. In *The Fourteenth International Conference on Learning Representations*, 2026.
- [26] Jiaheng Liu, Dawei Zhu, Zhiqi Bai, Yancheng He, Huanxuan Liao, Haoran Que, Zekun Wang, Chenchen Zhang, Ge Zhang, Jiebin Zhang, et al. A comprehensive survey on long context language modeling. *arXiv preprint arXiv:2503.17407*, 2025.
- [27] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhao Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for llm kv cache compression at test time. *Advances in Neural Information Processing Systems*, 36, 2024.
- [28] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2019.
- [29] Enzhe Lu, Zhejun Jiang, Jingyuan Liu, Yulun Du, Tao Jiang, Chao Hong, Shaowei Liu, Weiran He, Enming Yuan, Yuzhi Wang, Zhiqi Huang, Huan Yuan, Suting Xu, Xinran Xu, Guokun Lai, Yanru Chen, Huabin Zheng, Junjie Yan, Jianlin Su, Yuxin Wu, Neo Y. Zhang, Zhilin Yang, Xinyu Zhou, Mingxing Zhang, and Jiezhong Qiu. Moba: Mixture of block attention for long-context llms, 2025.
- [30] MAA. American invitational mathematics examination (aime). URL <https://maa.org/math-competitions/aime>, 2024.
- [31] Lingrui Mei, Jiayu Yao, Yuyao Ge, Yiwei Wang, Baolong Bi, Yujun Cai, Jiazhi Liu, Mingyu Li, Zhong-Zhi Li, Duzhen Zhang, et al. A survey of context engineering for large language models. *arXiv preprint arXiv:2507.13334*, 2025.
- [32] Shashi Narayan, Shay B. Cohen, and Mirella Lapata. Don’t give me the details, just the summary! Topic-aware convolutional neural networks for extreme summarization. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, Brussels, Belgium, 2018.

- [33] Dan Peng, Zhihui Fu, Zewen Ye, Zhuoran Song, and Jun Wang. Accelerating prefilling for long-context llms via sparse pattern sharing. *arXiv preprint arXiv:2505.19578*, 2025.
- [34] Dan Peng, Zhihui Fu, Zewen Ye, Zhuoran Song, and Jun Wang. Accelerating prefilling for long-context llms via sparse pattern sharing, 2025.
- [35] David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models, 2024.
- [36] Liliang Ren, Yang Liu, Yadong Lu, Yelong Shen, Chen Liang, and Weizhu Chen. Samba: Simple hybrid state space models for efficient unlimited context language modeling. *arXiv preprint*, 2024.
- [37] Noam Shazeer, *Azalia Mirhoseini, *Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.
- [38] Zecheng Tang, Quantong Qiu, Yi Yang, Zhiyi Hong, Haiya Xiang, Kebin Liu, Qingqing Dang, Juntao Li, and Min Zhang. Elastic attention: Test-time adaptive sparsity ratios for efficient transformers, 2026.
- [39] Zecheng Tang, Haitian Wang, Quantong Qiu, Baibei Ji, Ruoxi Sun, Keyan Zhou, Juntao Li, and Min Zhang. Loom-scope: a comprehensive and efficient long-context model evaluation framework. *arXiv preprint arXiv:2507.04723*, 2025.
- [40] Harsh Trivedi, Niranjana Balasubramanian, Tushar Khot, and Ashish Sabharwal. Musique: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.
- [41] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [42] Wenhao Wu, Yizhong Wang, Guangxuan Xiao, Hao Peng, and Yao Fu. Retrieval head mechanistically explains long-context factuality. *arXiv preprint arXiv:2404.15574*, 2024.
- [43] Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, Junxian Guo, Shang Yang, Haotian Tang, Yao Fu, and Song Han. Duoattention: Efficient long-context llm inference with retrieval and streaming heads. *arXiv*, 2024.
- [44] Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, Shang Yang, Haotian Tang, Yao Fu, Song Han, et al. Duoattention: Efficient long-context llm inference with retrieval and streaming heads. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [45] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations*, 2024.
- [46] Jing Xiong, Jianghan Shen, Fanghua Ye, Chaofan Tao, Zhongwei Wan, Jianqiao Lu, Xun Wu, Chuanyang Zheng, Zhijiang Guo, Min Yang, Lingpeng Kong, and Ngai Wong. UNComp: Can matrix entropy uncover sparsity? — a compressor design from an uncertainty-aware perspective. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 4179–4199, Suzhou, China, November 2025. Association for Computational Linguistics.
- [47] Peng Xu, Wei Ping, Xianchao Wu, Zihan Liu, Mohammad Shoeybi, and Bryan Catanzaro. Chatqa 2: Bridging the gap to proprietary llms in long context and rag capabilities. *arXiv preprint arXiv:2407.14482*, 2024.
- [48] Ruyi Xu, Guangxuan Xiao, Haofeng Huang, Junxian Guo, and Song Han. Xattention: Block sparse attention with antidiagonal scoring. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.
- [49] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025.

- [50] Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Yuxing Wei, Lean Wang, Zhiping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Zeng. Native sparse attention: Hardware-aligned and natively trainable sparse attention. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 23078–23097, Vienna, Austria, July 2025. Association for Computational Linguistics.
- [51] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. Big bird: Transformers for longer sequences. *Advances in neural information processing systems*, 33:17283–17297, 2020.
- [52] Chen Zhang, Yang Bai, Jiahuan Li, Anchun Gui, Keheng Wang, Feifan Liu, Guanyu Wu, Yuwei Jiang, Defei Bu, Li Wei, et al. Efficient context scaling with longcat zigzag attention. *arXiv preprint arXiv:2512.23966*, 2025.
- [53] Jintao Zhang, Chendong Xiang, Haofeng Huang, Jia Wei, Haocheng Xi, Jun Zhu, and Jianfei Chen. Spargeattn: Accurate sparse attention accelerating any model inference. In *International Conference on Machine Learning (ICML)*, 2025.
- [54] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark W. Barrett, Zhangyang Wang, and Beidi Chen. H2O: heavy-hitter oracle for efficient generative inference of large language models. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine, editors, *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023.
- [55] Weilin Zhao, Zihan Zhou, Zhou Su, Chaojun Xiao, Yuxuan Li, Yanghao Li, Yudi Zhang, Weilun Zhao, Zhen Li, Yuxiang Huang, Ao Sun, Xu Han, and Zhiyuan Liu. Infiln-v2: Dense-sparse switchable attention for seamless short-to-long adaptation, 2025.

A Code & Model

We open-source our code and model as follows: <https://github.com/qqtang-code/FluxAttention>.

B Related Work

B.1 Sparse Attention Mechanisms

To mitigate the quadratic complexity of standard attention mechanisms, existing research has broadly advanced along two trajectories: inference-time heuristics and training-aware sparsification. Inference-time heuristics typically employ static patterns, such as fixed sliding windows or strides [45, 14, 3], to restrict the receptive field. To capture dynamic dependencies more effectively, content-aware approaches have been proposed. For instance, token eviction policies discard uninformative tokens based on accumulated importance scores [54, 23, 27], whereas kernel-based estimators identify salient blocks to bypass redundant computations [19]. Complementarily, prefill optimizers leverage importance-driven selection to accelerate the processing of long contexts [21, 48, 53, 34, 18]. Despite the effectiveness of these heuristic methods, they frequently rely on sensitive hyperparameters, thereby limiting their robustness across diverse tasks.

In contrast, training-aware sparsification internalizes sparsity within the optimization objective to align the training process with sparse inference. A prominent direction in this area involves learnable selection. For instance, SeerAttention [10], NSA [50], and MoBA [29] employ learnable gates and hierarchical constraints to approximate ground-truth attention patterns. To bridge the gap between dense pre-training and sparse adaptation, InfLLM-v2 [55] introduces a dense-sparse switchable mechanism via parameter-free pooling, whereas DSA [8] utilizes a lightning indexer alongside a two-stage training strategy to efficiently filter the top- k key-value pairs. However, the majority of these methods focus on fine-grained, block-level or token-level selection within a fixed attention framework, rather than dynamically adapting the overarching attention mode itself based on input complexity.

B.2 Hybrid Architectures and Dynamic Allocation

To balance computational efficiency and model performance, hybrid architectures strategically integrate Full Attention (FA) with linear-complexity operators. The dominant paradigm, inter-layer hybridization, interleaves linear layers with standard attention layers to recover associative recall capabilities [20, 7]. Notable large-scale implementations, such as Jamba [24], utilize fixed block-wise ratios, whereas variants optimize memory utilization through shared global blocks [11] or sliding windows [36]. More recently, intra-layer hybridization has emerged as a strategy to refine structural granularity. For example, PruLong [4] and DuoAttention [43] combine FA and Sparse Attention (SA) within individual layers by assigning different attention heads to different computational modes. Furthermore, LongCat [52] proposes the LoZA mechanism, constructing a static ZigZag topology by replacing low-sensitivity Multi-head Latent Attention (MLA) modules with linear-complexity SA. A critical limitation of these approaches is their reliance on static topologies or pre-defined ratios established prior to inference, lacking the flexibility required to dynamically distinguish diverse tasks.

To address the rigidity of static designs, recent studies have explored dynamic allocation strategies. For instance, Elastic Attention [38] dynamically allocates varying sparsity at the head level based on contextual importance. While offering algorithmic flexibility, such head-level dynamic sparsity introduces severe hardware inefficiencies. Specifically, varying context lengths across different attention heads lead to severe synchronization bottlenecks, as fast-executing sparse heads must wait for memory-intensive retrieval heads within the same layer. This creates significant memory bandwidth bottlenecks, severely hindering hardware acceleration and limiting practical speedups, especially during the autoregressive decoding phase.

B.3 Dynamic Routing in Neural Networks

Dynamic routing and conditional computation have long been studied to decouple model capacity from inference cost. Traditional approaches, such as Mixture-of-Experts (MoE) [37, 9], effectively route tokens to specialized Feed-Forward Network (FFN) experts. Recent advancements like Mixture-of-Depths (MoD) [35] extend this concept by dynamically skipping specific layers for uninformative tokens to optimize compute allocation.

While these methods successfully route computation dynamically, they predominantly focus on FFNs or complete layer-skipping, leaving the dynamic optimization of the attention mechanism itself largely underexplored. Unlike fine-grained or head-level allocation schemes that disrupt memory continuity, our proposed **Flux Attention** introduces a context-aware, layer-level routing mechanism. By utilizing a lightweight Layer Router to dynamically toggle entire layers between FA and SA, our approach bridges the gap between context-aware algorithmic flexibility and hardware-friendly contiguous memory access, translating theoretical computational reductions into substantial wall-clock speedups.

C Sparsification Setup and Latency Profiling Implementation

This section details the layer importance identification, the progressive sparsification strategy, and the hardware latency measurement protocol mentioned in Section 2.3.

C.1 Layer Entropy Score Calculation

Following the methodology proposed by UnComp [46], we identify and rank Transformer layers based on their informational density and uncertainty when processing long contexts. We use a matrix entropy-based profiling method, quantifying the information content of each layer over long-context validation datasets to estimate its inherent structural sparsity.

For a given layer ℓ , we calculate its Entropy Score (E_ℓ) by measuring the truncated matrix entropy of its hidden representations. Formally, let s be the input sequence length, d be the hidden dimension, and $\mathcal{X}^{(\ell)} \in \mathbb{R}^{s \times d}$ be the hidden states matrix of layer ℓ . We first derive the trace-normalized covariance matrix $\Sigma^{(\ell)} = \frac{\mathcal{X}^{(\ell)}(\mathcal{X}^{(\ell)})^\top}{\text{Tr}(\mathcal{X}^{(\ell)}(\mathcal{X}^{(\ell)})^\top)}$. The score is computed as the von Neumann entropy over its top- K eigenvalues:

$$E_\ell = - \sum_{i=1}^K \lambda_i^{(\ell)} \log \lambda_i^{(\ell)} \quad (7)$$

where $\lambda_i^{(\ell)}$ denotes the i -th largest eigenvalue of $\Sigma^{(\ell)}$, and K is the truncation threshold used to filter out noise. A lower E_ℓ indicates lower information density (i.e., lower uncertainty) and higher redundancy, making the layer a suitable candidate for sparsification.

C.2 Progressive Sparsification Strategy

Based on the computed entropy scores E_ℓ , we evaluate the information density of all L layers across the model. As defined in the main text, the Model Sparsity Ratio (Ω_{MSR}) represents the proportion of layers converted to sparse attention. To simulate the varying levels of sparsity reported in our experiments (e.g., $\Omega_{\text{MSR}} = 20\%$), we use a thresholding mechanism based on these scores. We first determine the number of layers to preserve as full attention via $k = \lfloor (1 - \Omega_{\text{MSR}}) \cdot L \rfloor$. The k layers with the highest entropy scores are retained as retrieval layers to ensure global information integration and preserve complex contextual pathways. The remaining $(L - k)$ layers with the lowest entropy scores are replaced with sparse layers.

C.3 Latency Measurement Implementation

To evaluate the hardware efficiency of different sparsity paradigms, we profile latency during the autoregressive decoding phase. All latency measurements are performed on a single NVIDIA A800 GPU (80GB) using PyTorch with BF16 precision.

To simulate realistic long-context retrieval scenarios while isolating the decoding bottleneck, we fix the batch size to 1 and evaluate across varying prompt sequence lengths. For each configuration, we perform 10 warm-up steps to initialize the CUDA context and stabilize GPU clocks, followed by 50 profiling iterations. The reported latency is the average wall-clock time required to generate a single token.

Implementation of Sparsity Baselines For the head-level sparsity baseline, we retain a subset of attention heads for dense computation while the remaining heads operate sparsely. However, highly optimized attention kernels (e.g., FlashAttention) lack hardware-level support for processing mixed context lengths across different heads within the same layer. Consequently, enforcing head-level sparsity results in fragmented, non-contiguous memory access patterns. The GPU memory bandwidth is still consumed by loading the full historical KV cache into SRAM, leading to only marginal wall-clock speedups despite the theoretical FLOP reduction.

In contrast, our layer-level sparsity implementation avoids this issue. When a layer operates sparsely, the decoding step fetches only the locally required KV states, bypassing the global historical KV tensors. This layer-level routing allows contiguous memory loading, translating theoretical sparsity into proportional decoding acceleration. We calculate the speedup as the ratio of the latency of the full dense model to that of the sparsified model for a given input length.

D Implementation Details

This section details the training configurations, baseline implementations, and system-level optimizations for efficient long-context processing.

Table 3: Hyperparameters: General configuration.

Hyperparameter	Value
<i>Model & Training</i>	
Base Model	Qwen, Llama
Sequence length	65536
Precision	bfloat16
Global Batch Size	48
Training Steps	300
Mask / Reg. LR	$5e^{-4} / 1e^{-3}$
Warmup Ratio	0.2
AdamW Momentum (β_1, β_2)	(0.9, 0.95)
Weight Decay	0.1
Learning Rate Schedule	Cosine
<i>Sparsity Config</i>	
Pool Size	100
Sink / Local Size	128 / 2048
Block / Chunk Size	64 / 16384
Stride / Threshold	16 / 0.9
Selection Mode	Inverse

D.1 Training Configuration and Hyperparameters

We evaluate the proposed approach on models of various sizes, including Qwen3-4B, Qwen3-8B [49], and Meta-Llama-3.1-Instruct [12]. We freeze the pre-trained backbone and update only the parameters of the Layer Router to maintain the general capabilities of the model. For task representation, we apply a Prefill-Suffix Pooling operation to aggregate the first 100 and the last 100 tokens of the sequence, as these segments typically contain the system instructions and user queries required to identify the task.

We train all models with a sequence length of $L = 65,536$ tokens in bfloat16 precision using the AdamW optimizer [28] ($\beta_1 = 0.9, \beta_2 = 0.95$). Training is conducted on a distributed cluster with Fully Sharded Data Parallel (FSDP) under a hybrid sharding strategy. To balance the convergence of the router and sparsity regularization, we apply a decoupled learning rate schedule. The Layer Router uses a learning rate of 5×10^{-4} for rapid adaptation to retrieval patterns, while the sparsity regularization terms use a higher learning rate of 1×10^{-3} . The dual regularization coefficients λ_1 and λ_2 are randomly initialized and optimized alongside the router parameters. A cosine decay learning rate schedule is applied after a linear warmup phase over the first 20% of the training steps.

D.2 Baseline Implementation Details

We compare the proposed approach with several state-of-the-art sparse attention mechanisms, categorizing them into training-free and training-based methods. For training-free baselines, we evaluate TriangleMix² [14], which relies on heuristic-based sparsity without parameter updates. For training-based baselines, including PruLong³ [4] and DuoAttention⁴ [44], we follow a unified fine-tuning protocol. We train all baselines in identical environments and on the same dataset while maintaining their original hyperparameter settings.

D.3 Sparsity and Kernel Configuration

We use Block-Sparse-Attention [13] for efficient streaming inference to control the granularity and retention policy of the attention mechanism. We set the block size to 64 to define the minimum unit of sparsity, and the chunk size to 16,384 to process ultra-long sequences. A sink token size of 128 is maintained to preserve the attention sink phenomenon, ensuring stability during streaming generation. Additional kernel parameters, such as stride, normalization, and selection modes, are detailed in the Sparsity Config section of Table 3.

²<https://github.com/microsoft/MInference/tree/main/TriangleMix>

³<https://github.com/princeton-plt/PruLong>

⁴<https://github.com/mit-han-lab/duo-attention>

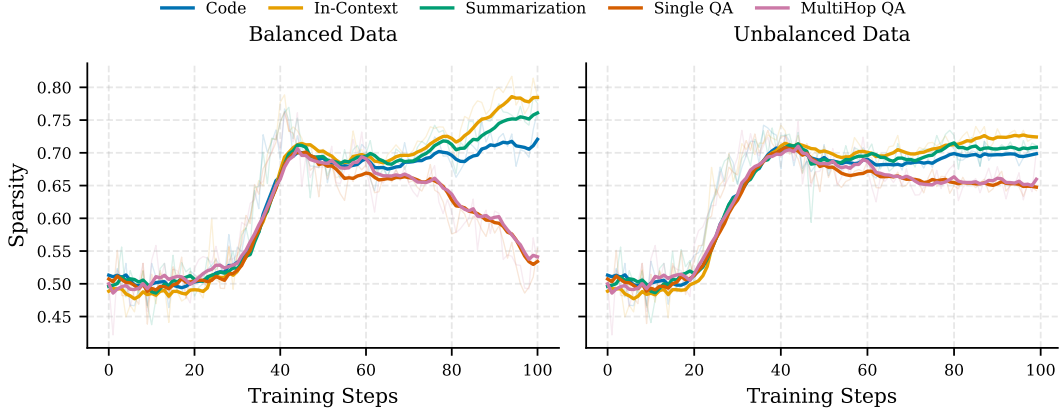


Figure 7: Evolution of sparsity levels across training steps under different data distributions. **Left:** Training on a well-balanced dataset, where the router successfully disentangles tasks into distinct sparsity levels. **Right:** Training on an unbalanced dataset dominated by context-holistic tasks, leading to homogenized routing.

E Analysis

E.1 Impact of Data Composition on Task Differentiation

In previous sections, we have established that Flux Attention dynamically tailors sparsity to specific task demands. To fully unleash this capability, we discover that a well-balanced training curriculum acts as a crucial catalyst. To empirically validate this, we analyze the routing dynamics—specifically, the evolution of sparsity levels across training steps—under different data distribution settings.

Figure 7 (Left) illustrates the sparsity trajectories when the router is trained on a well-balanced dataset. Driven by this diverse curriculum, the router successfully disentangles the underlying task demands and exhibits a clear divergence in its routing behavior. Notably, after an initial shared exploration phase, retrieval-intensive tasks converge to a lower sparsity level to preserve critical historical keys and values. In contrast, context-holistic tasks confidently sparsify the context, diverging toward higher sparsity levels. This demonstrates that a balanced mixture effectively teaches the router to establish robust, task-specific boundaries.

Conversely, Figure 7 (Right) demonstrates the routing behavior when the training data is heavily skewed (e.g., dominated by context-holistic tasks). Under this setting, the router faithfully optimizes for the predominant data distribution. Rather than maintaining distinct task boundaries, the sparsity trajectories fail to clearly diverge after the initial phase, naturally converging toward a shared target sparsity. This results in a more homogenized routing strategy tailored to the specific domain it was exposed to.

This analysis yields an important insight into the training dynamics of the Layer Router: the router intrinsically aligns its allocation strategy with the global optimization landscape provided by the training data. Therefore, to train a general-purpose model capable of fine-grained, context-aware sparsification across diverse tasks, constructing a balanced task mixture during training is the optimal and highly effective practice.

E.2 Impact of Input Truncation on Task Identification

To optimize the trade-off between routing efficiency and accuracy, we investigate the sensitivity of the layer router to the input sequence length. Specifically, we analyze how varying the truncation budget influences the capacity of the router to distinguish between task types and allocate appropriate sparsity patterns. Figure 8 illustrates the performance and sparsity trends as the pooling window expands from 50 tokens (boundary-only) to the full sequence.

Our default strategy extracts only the first and last 100 tokens. This design leverages the structure of long-context prompts, where task-defining instructions typically appear at the beginning of the sequence, and specific user queries are appended at the end. The intermediate content primarily consists of raw context. Although this context is necessary for generation, it acts as noise during the routing process, which focuses on macro-level task identification.

Contrary to the assumption that additional context improves routing, Figure 8 demonstrates a drop in performance when the pooling size exceeds 100 tokens. We attribute this phenomenon to the limited capacity of the lightweight MLP within the routing module. As the pooling window expands, the task identification signals are diluted by the

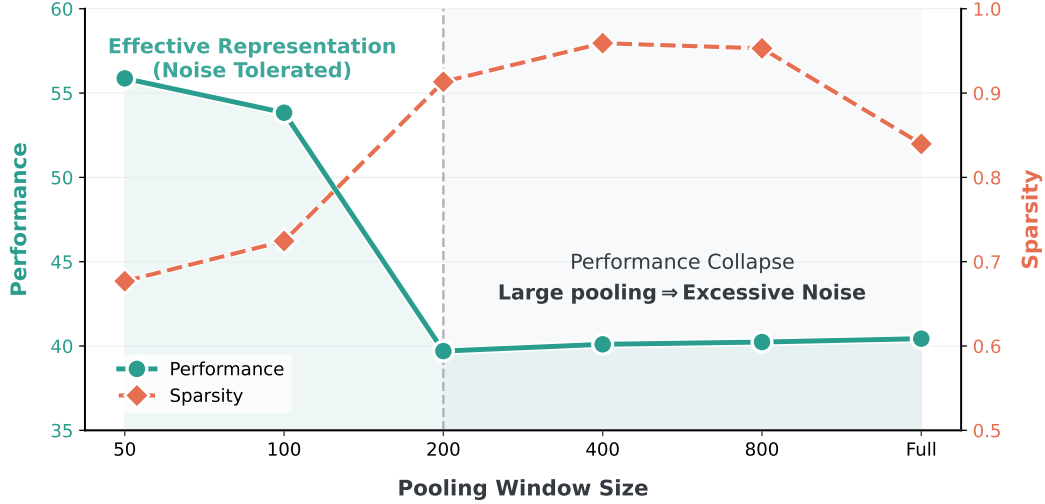


Figure 8: Impact of pooling window size on downstream performance and routing sparsity (Ω_{MSR}). We evaluate varying truncation budgets ($L \in \{50, 100, 200, 400, 800, \text{Full}\}$), retaining only the sequence boundaries (prefix and suffix). Increasing the pooling size beyond 100 tokens introduces context noise, which disrupts the routing mechanism. Consequently, the router misclassifies task features and assigns excessive sparsity to retrieval-intensive tasks, thereby degrading the overall performance.

document tokens. The MLP struggles to filter out this noise and fails to capture the semantic features necessary for classification. Consequently, the router makes suboptimal decisions, such as assigning high sparsity levels (> 0.9) to retrieval-intensive tasks that require denser attention. This misallocation causes the observed decrease in the quality of generation. These findings support the choice of a 100-token boundary window to maintain an optimal signal-to-noise ratio and facilitate accurate feature extraction.

E.3 Loss Curves and Performance Metrics

We examine the training stability and dynamic routing behavior of Flux Attention by visualizing the optimization dynamics in Figure 10. This analysis decomposes the training process into the primary language modeling loss, the sparsity regularization loss, the evolution of the routed sparsity metric (Ω_{MSR}), and the adaptive coefficients (λ).

Optimization Stability. As shown in Figures 10a and 10b, the joint optimization of the language modeling objective and Layer Router parameters remains stable. The LM loss decreases rapidly and plateaus around 1.8, suggesting that the lightweight Layer Router and the introduced sparsity do not impede convergence. Meanwhile, the sparsity regularization loss drops significantly within the first 100 steps. This indicates that the continuous relaxation scheme via Gumbel-Softmax effectively guides the router toward the specified sparsity constraints.

Differentiation in Flux Attention Allocation.

Figure 10c provides empirical support for our motivation in Section 1, showing that downstream tasks exhibit varying sensitivities to attention sparsity. Starting from a neutral initialization, the Layer Router learns to differentiate between task types automatically. Retrieval-intensive tasks converge to higher Ω_{MSR} values, representing a larger allocation of Full Attention to preserve performance. In contrast, context-holistic tasks stabilize at lower values near

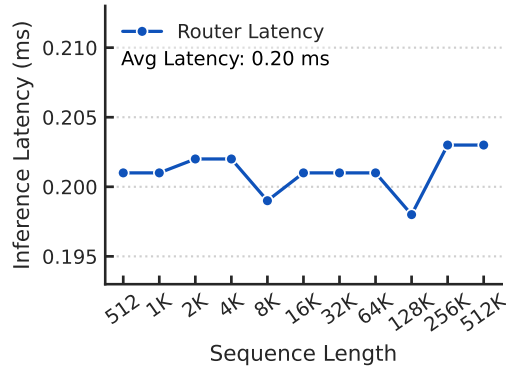


Figure 9: **Router latency analysis.** The router incurs negligible overhead (avg. 0.20 ms). Our design ensures length-invariant stability, maintaining constant speed from 512 to 1M tokens.

the target threshold. This confirms that Flux Attention identifies tasks capable of tolerating higher sparsity, thereby improving inference throughput without redundant computation.

Adaptive Coefficients. Figure 10d tracks the evolution of the Lagrangian multipliers (λ), which dynamically scale the penalty for sparsity violations. We observe that λ increases most aggressively for certain tasks, suggesting the model prioritizes meeting density requirements where necessary. This adaptive mechanism balances the trade-off between computational cost and model quality automatically, eliminating the need for manual, task-specific tuning.

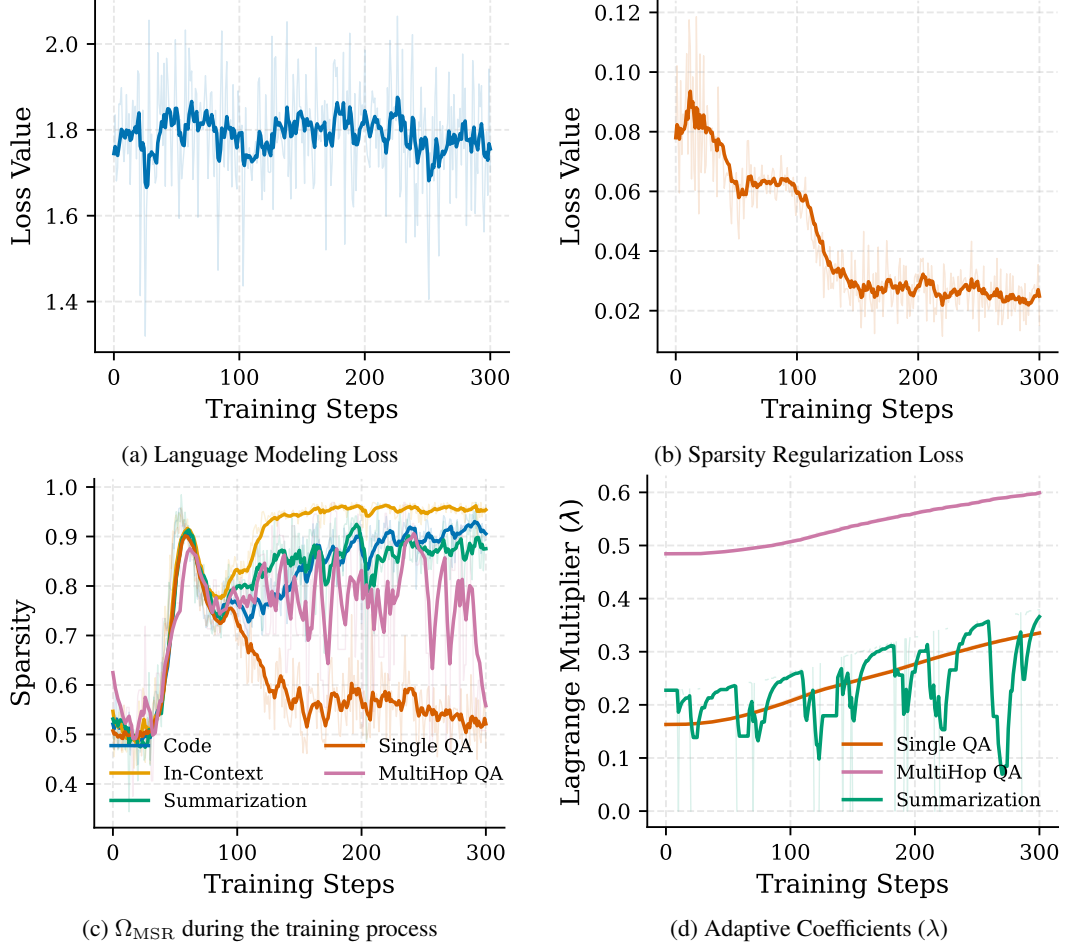


Figure 10: Decomposition of Training Objectives for Flux Attention. We visualize the training dynamics of the Layer Router, separating the total loss into (a) the primary language modeling objective and (b) the sparsity regularization term. Subfigures (c) and (d) illustrate the task-level differentiation in sparsity allocation (Ω_{MSR}) and adaptive coefficients (λ), demonstrating how the model automatically distinguishes between context-holistic and retrieval-intensive tasks.

F Error Analysis

In Table 11, 12, and 13, we present representative model outputs comparing our method with other baselines. Due to the extensive length of the contexts, only a partial input context is shown. We observe that the primary source of performance improvement stems from our method’s ability to accurately identify and respond to the key contextual segments relevant to the query.

Case 1: Reading Comprehension & Information Extraction

Context:

"Low-cost solutions can give billions access to modern cooking by 2030, but the world is failing to deliver... Nearly one in three people around the world still cook their meals over open fires or on basic stoves, resulting in significant damage to health, living standards and gender equality – and yet this challenge can be overcome this decade through a relatively modest amount of investment... Today, 2.3 billion people rely on charcoal, firewood, coal, agricultural waste and animal dung as fuel to prepare meals, causing them to breathe in harmful smoke in the process..."

Question:

Based on the information provided in these news articles, which of the following conclusions regarding African clean cooking is the most reliable?

Correct Prediction (Ours / Ground Truth):



Option B

Content: **In comparison to other regions of the world, the issue of outdated cooking methods in Africa is particularly severe:** nearly four-fifths of the African population still rely on traditional stoves or open fires for cooking, a figure that stands at less than one half globally.

Incorrect Baselines:

Base Model (Qwen3-4B), PruLong, DuoAttention, Triangle:

Pred: Option A

Content: **African nations broadly benefit from carbon markets, particularly in addressing financing issues for clean cooking:** Carbon credits can bridge the funding gap for clean cooking investments in Africa, while also enhancing the affordability of clean cookstoves and fuels.

(Error: Baselines hallucinate or improperly infer information about carbon markets, failing to ground their conclusion strictly in the provided text's severity statistics.)

Figure 11: **Comparison on a long-context reading comprehension task.** Our model accurately extracts and verifies the severity statistics of outdated cooking methods in Africa compared to global figures, while all baselines consistently fall for the same unsupported distractor regarding carbon markets.

Case 2: Core Argument Identification in Abstract Text

Context:

"The Lawyer as Friend: The Moral Foundations of the Lawyer-Client Relation" Charles Friedt Advocatus
sed non ladro, Res miranda populo Medieval anthem honoring St. Ives

Can a good lawyer be a good person? The question troubles lawyers and law students alike. They are troubled by the demands of loyalty to one's client and by the fact that one can win approval as a good, maybe even great, lawyer even though that loyalty is engrossed by over-privileged or positively distasteful clients. How, they ask, is such loyalty compatible with that devotion to the common good characteristic of high moral principles? And whatever their views of the common good, they are troubled because the willingness of lawyers to help their clients use the law to the prejudice of the weak or the innocent seems morally corrupt..."

Question:

What is the core argument of this article?

Correct Prediction (Ours / Ground Truth):



Option C

Content: **Refuting the social doubts about lawyers' professional ethics through analogy.**

Incorrect Baselines:

Base Model (Llama3.1-8B), DuoAttention, Triangle:

Pred: Option B Content: **A good lawyer can be a good person.**

(Error: Baselines fail to abstract the overarching argumentative framework, instead parroting the literal opening rhetorical question as the core argument itself.)

PruLong:

Pred: Option A Content: **Lawyers should be regarded as friends of clients.**

(Error: The model superficially extracts the title text ("The Lawyer as Friend") without understanding that the "friend" concept is merely the analogy used to resolve the broader ethical doubts.)

Figure 12: **Qualitative comparison on identifying the core argument in a philosophical legal text.** Our model successfully synthesizes the text to identify the underlying argumentative strategy (refutation via analogy), whereas baselines are easily distracted by literal sentences from the title and opening hook.

Case 3: Methodology Extraction from Academic Paper

Context:

"Published as a conference paper at ICLR 2024 MAGICDRIVE: STREET VIEW GENERATION WITH DIVERSE 3D GEOMETRY CONTROL ABSTRACT Recent advancements in diffusion models have significantly enhanced the data synthesis with 2D control. Yet, precise 3D control in street view generation, crucial for 3D perception tasks, remains elusive. Specifically, utilizing Bird's-Eye View (BEV) as the primary condition often leads to challenges in geometry control (e.g., height), affecting the representation of object shapes, occlusion patterns, and road surface elevations... In this paper, we introduce MAGICDRIVE, a novel street view generation framework, offering diverse 3D geometry controls including camera poses, road maps, and 3D bounding boxes, together with textual descriptions, achieved through tailored encoding strategies..."

Question:

How does the MagicDrive encode the bounding box in training step?

Correct Prediction (Ours / Ground Truth):



Option D

Content: MagicDrive encode the class labels and the corner points separately, and then uses an MLP to compress them into a vector.

Incorrect Baselines:

Base Model (Qwen3-8B), PruLong, DuoAttention, Triangle:

Pred: Option B Content: MagicDrive uses Fourier embedding for 4 corner point and passes them through an MLP for encoding in training step. MagicDrive then uses an MLP to compress both the class label and position embeddings into a single hidden vector for each bounding box.

(Error: Baselines hallucinate or misattribute specific technical details, such as "Fourier embedding for 4 corner points," failing to accurately extract the exact encoding mechanism described in the paper.)

Figure 13: **Qualitative comparison on extracting technical methodology from a machine learning paper.** Our model accurately identifies the specific bounding box encoding strategy, whereas all baselines suffer from hallucination, confidently generating plausible but incorrect architectural details (Fourier embeddings) not supported by the text.